

09 · Compaction 的触发、切点、摘要与上下文不变量

上下文窗口接近上限时 Pi 如何压缩历史，并保持工具调用关系可重放

分析版本：027a5847901b5dde30270abaa1041046cd2b4b55

核心结论

自动 Compaction 的触发公式是 $\text{contextTokens} > \text{contextWindow} - \text{reserveTokens}$ ；默认 $\text{reserveTokens}=16384$ 、 $\text{keepRecentTokens}=20000$ 。切点从尾部反向累计估算 token，但只允许落在 `user/assistant/bash/custom/summary` 等可见消息上，明确禁止 `toolResult`，保证 `tool result` 不会脱离它前面的 `tool call`。

1. 触发阈值为什么预留 `reserveTokens`

如果等到 `contextWindow` 完全耗尽才压缩，就没有空间生成下一条 `assistant` 回复。`shouldCompact` 因此用 $\text{contextWindow} - \text{reserveTokens}$ 作为阈值。`reserveTokens` 是“为下一次输出预留的安全空间”，不是要保留的历史大小；`keepRecentTokens` 才决定压缩后大约保留多少最近上下文。

2. token 估算优先使用真实 `usage`

`estimateContextTokens` 先找最后一个有效 `assistant usage`：它通常比 `chars/4` 估算可靠。`usage` 之后尚未被下一次模型调用计入的 `trailing messages` 再用 `estimateTokens` 估算。若没有任何有效 `usage`，才对全部消息启发式估算。`error/aborted/all-zero usage` 不被当作可靠锚点。

3. `findCutPoint` 的核心算法

1. 确定可考虑的 `entry` 区间（考虑上次 `compaction boundary`）。
2. 枚举 `valid cut points`。
3. 从最新 `entry` 向前累计每条 `context message` 的估算 token。
4. 达到 `keepRecentTokens` 后，选择当前位置及之后最近的 `valid cut point`。
5. 向前吸收紧邻的非 `context metadata`，避免无意义切碎。
6. 判断切点是否位于一个超大 `turn` 中间，必要时进入 `split-turn` 模式。

4. 为什么绝不能切在 `toolResult`

`toolResult` 的语义依赖前面 `assistant` 中的 `toolCallId/name`。若摘要切点把 `toolResult` 留在新上下文，却把发起它的 `assistant toolCall` 总结掉，`Provider` 在历史重放时可能看到“孤儿 `tool result`”，违反 `OpenAI/Anthropic` 等协议配对要求。源码的 `isCutPointMessage` 对 `toolResult` 明确返回 `false`。

5. 为什么可以切在带 toolCall 的 assistant 上

若切点是 assistant，后续 toolResult 位于它之后，因此二者一起保留。这个方向是安全的：保留 call + results，而不是保留 results 丢 call。Compaction 文档也明确把 assistant 视为合法切点。

6. Split Turn：单个 turn 本身就超过 keepRecentTokens

正常希望按完整 turn 切，但若一个 user turn 经过大量工具调用就超过 20k，切点可能落在本 turn 的 assistant 中。算法向前找到 turnStartIndex，把早期 turn prefix 单独摘要，再与历史 summary 合并，保留后半段。否则只能保留整个巨型 turn，压缩失效。

7. 摘要不是把 Session 历史删掉

Compaction append 一个 CompactionEntry；原始 JSONL 历史仍在树上。改变的是 buildContextEntries 的投影：模型后续只看到 compactionSummary + kept tail。这样可以继续审计原始会话，又能缩短模型上下文。

8. 重复 Compaction 如何累计

prepareCompaction 会找到上一 compaction，并将 previousSummary 纳入下一次摘要。文件读写信息也从 previous compaction.details 与本次待总结消息中的 tool calls 累积。这避免每轮压缩都丢失早期“读过哪些文件、修改过哪些文件”的重要状态。

9. 摘要请求本身的格式

待总结消息先 convertToLlm，再 serializeConversation 成普通文本，使用 [User]、[Assistant]、[Tool result] 等标签。这样摘要模型看到的是“要总结的数据”，而不是一段可继续执行工具的真实对话。工具结果还会截断，以控制摘要请求成本。

10. AgentSession 中的触发时机

新用户 prompt 前会检查上一条 assistant，Agent run 结束后也会在 post-run 处理 retry/compaction。Compaction 有 manual、threshold、overflow 三类 reason；overflow recovery 可能在上下文溢出错误后压缩再重试。

11. 扩展为什么能接管 Compaction

session_before_compact 事件拿到 preparation、branchEntries、reason、willRetry、signal。扩展可以 cancel 或提供自定义 CompactionResult；因此“何时切、切哪里”的基础计算与“如何总结”可以分离。自定义实现仍应尊重 firstKeptEntryId/tokensBefore 等结构不变量。

12. 风险与测试重点

- 切点若落在 toolResult，会造成 Provider replay 协议错误。

- 上一 compaction 的 firstKeptEntryId 不在当前 branch 时必须有 fallback。
- tokensBefore 应以真正 pre-compaction context 重算，不能简单复用旧 usage。
- 摘要生成失败/取消不应破坏 session leaf 或丢失旧历史。
- overflow recovery 必须防止无限“压缩 -> 仍溢出 -> 压缩”循环。
- Compaction usage 应计入 session 总成本，否则统计低估。

动态验证方案

实验 1：阈值测试

假设：shouldCompact 只在超出 window-reserve 时为 true

操作：对固定 contextWindow=100000, reserve=16384 测 83616 与 83617。

预期：前者 false，后者 true（严格大于）。

若不一致：若边界不同，检查版本或设置覆盖。

实验 2：工具配对切点

假设：findCutPoint 不会返回 toolResult entry

操作：构造 user->assistant(toolCall)->toolResult 的长序列，调 findCutPoint。

预期：firstKeptEntry 落 user/assistant 等合法点，不落 toolResult。

若不一致：若落 toolResult，这是高风险回归，应立即加测试。

实验 3：恢复模型

假设：Compaction 后 buildSessionContext 只投影 summary + kept tail

操作：执行 /compact 后读取 JSONL 与 buildContextEntries。

预期：原历史仍在文件；LLM context 省略被总结段。

若不一致：若原文件被删除或上下文仍全量，检查 SessionManager checkpoint 逻辑。

源码证据索引

文件	关键符号/用途	本报告结论
packages/coding-agent/src/core/compaction/compaction.ts	DEFAULT_COMPACTON_SETTINGS / shouldCompact / estimateContextTokens / findCutPoint	触发与切点核心算法
packages/coding-agent/src/core/compaction/utils.ts	serializeConversation / file tracking	摘要输入与累计状态
packages/coding-agent/src/core/agent-session.ts	_checkCompaction / manual/overflow/retry	运行时触发编排
packages/coding-agent/src/core/session-manager.ts	CompactionEntry / buildContextEntries	持久化与恢复投影

packages/coding-agent/docs/compaction.md	算法、split turn、extension hooks	行为文档交叉验证
packages/coding-agent/test/agent-session-compaction.test.ts	回归/边界测试	行为契约

证据状态：除非单独标注，本报告中的行为结论均来自上述固定 commit 的静态源码与仓库测试/文档交叉验证；未实际启动 Pi、未抓取真实网络请求，因此“运行结果已验证”项为空。

推荐阅读顺序

优先级	文件	阅读目的	精读
1	packages/coding-agent/src/core/compaction/compaction.ts	从 shouldCompact 精读到 findCutPoint	是
2	packages/coding-agent/src/core/session-manager.ts	理解压缩后的上下文重建	是
3	packages/coding-agent/src/core/agent-session.ts	看自动触发/重试编排	是
4	packages/coding-agent/src/core/compaction/utlis.ts	看摘要序列化和文件跟踪	否
5	packages/coding-agent/docs/compaction.md	用图示核对 split turn 心智模型	是